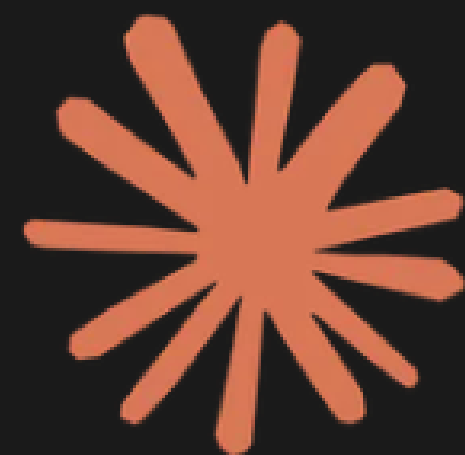




Trilha

CLAUDE CODE

O PRÓXIMO PASSO DA SUA CARREIRA

Aula 6 — Claude Code pra dados. Da planilha suja ao agente de IA.



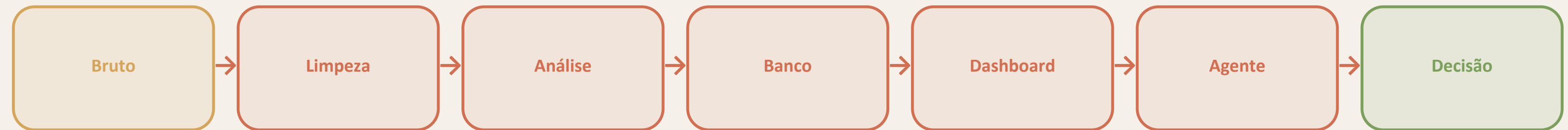
Dado bruto não vale nada. Até virar *decisão*.

E entre o bruto e a decisão tem 6 estações. Cada uma com seu erro caro.



A *cadeia*. Bruto → decisão.

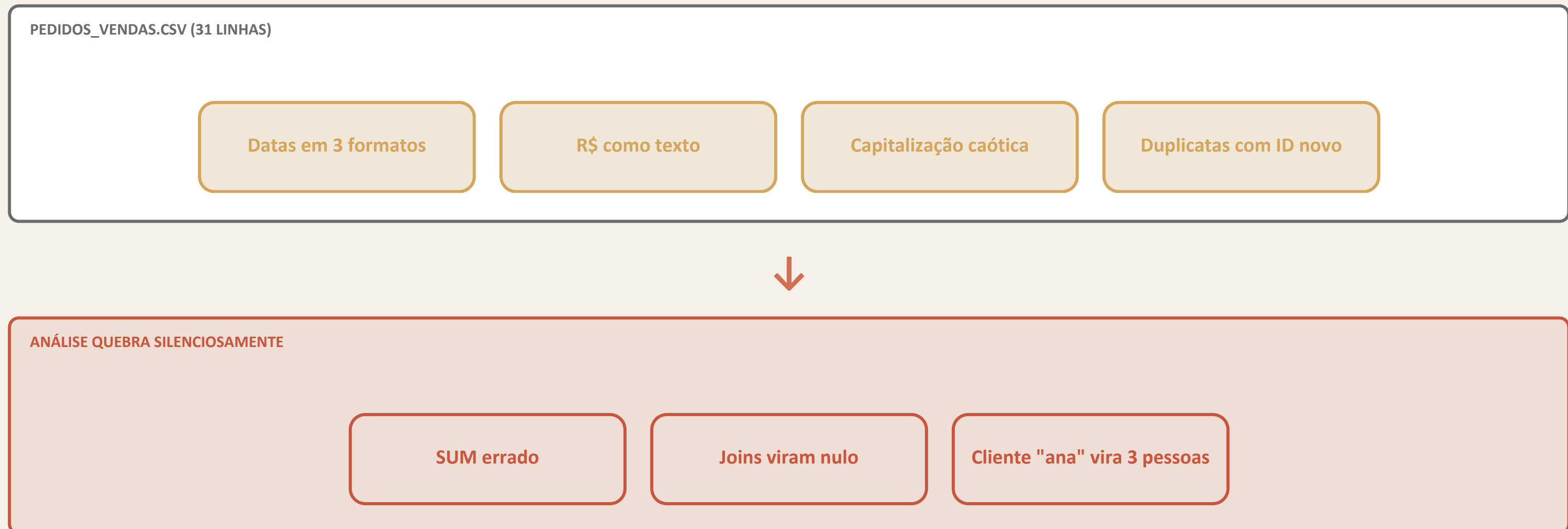
6 estações. Cada uma entrega um artefato que a próxima consome.





Problema #1 *Planilha suja*. Erro silencioso.

Datas em 3 formatos, R\$ como texto, duplicatas com ID novo — análise sai errada e ninguém vê.





Problema #2 *Scripts soltos*. Análise não vira produto.

Notebook abre, gráfico sai, screenshot vai pro Slack — e o dado morre ali.

NOTEBOOK DO ANALISTA

pandas.read_csv

groupby

matplotlib

screenshot pro Slack



RESULTADO

Análise morre no chat

Diretor não acessa

Nada vira produto



Problema #3 *Insight invisível*. Falta linguagem natural.

Dashboard responde só perguntas que o analista já antecipou. Pergunta nova → 2 dias.





Você deixa de *digitalar*. Passa a *dirigir*.

A habilidade nova: especificar bem, dar contexto, validar com rigor.



Analista que *digita*. Analista que *dirige*.

Mesma pessoa, mesma stack, escalas diferentes.

ANALISTA QUE DIGITA

Operação artesanal

- ✗ Reescreve a limpeza a cada planilha
- ✗ Análise vive em Jupyter solto
- ✗ Dashboard é projeto de semana
- ✗ Pergunta nova → 2 dias
- ✗ Cresce mas não escala

ANALISTA QUE DIRIGE

Operação industrializada

- ✓ Limpeza é script reprodutível
- ✓ Análise tem contexto vivo (.llm/, CLAUDE.md)
- ✓ Dashboard sai numa tarde
- ✓ Pergunta nova → 30 segundos no Telegram
- ✓ Mesma cabeça, 10x mais entrega



Os *6 blocos* da aula.

Cada um entrega um artefato. Cada um valida o anterior.

1. Limpeza
20 min · pandas + Claude

2. Análise
25 min · 3 frentes

3. Banco
20 min · Supabase MCP

4. Dashboard
25 min · Streamlit + UI

5. Deploy
10 min · Streamlit Cloud

6. Agente
25 min · Telegram + API

Setup
pré-aula

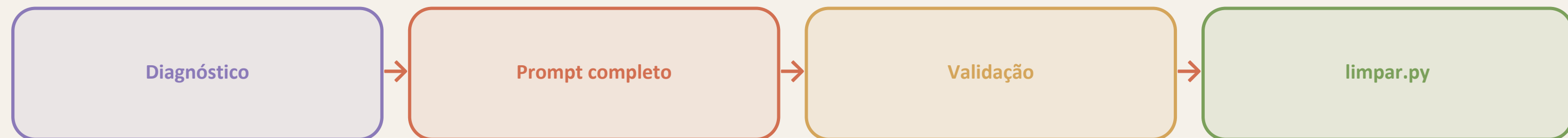
Demo case
e-commerce real

Validação
após cada bloco



Bloco 1 *Limpeza*. Diagnóstico, prompt, validação.

O entregável não é o CSV. É o limpar.py — reprodutível.



O QUE O PROMPT COMPLETO DESCREVE

- ✓ Title Case em nomes e região (vazia → "Não informado")
- ✓ Datas pra AAAA-MM-DD (3 formatos: DD/MM/AAAA, AAAA-MM-DD, "D Mês AAAA")
- ✓ Receita pra decimal (sem R\$, sem espaços)
- ✓ Status normalizado (PT/EN → "Fechado Ganho"/"Fechado Perdido")
- ✓ Coluna suspeita_duplicata (mesmo cliente, data, receita, ID diferente)



Bloco 2 *Análise*. Contexto + 3 frentes.

CLAUDE.md vira a system message do projeto. Sem ele, você reexplica a cada sessão.

SETUP DO CONTEXTO

- ✓ mkdir case-01-analise/.llm
- ✓ cp database.md .llm/ (schemas das tabelas)
- ✓ /init → edita CLAUDE.md com stack, dados, regras



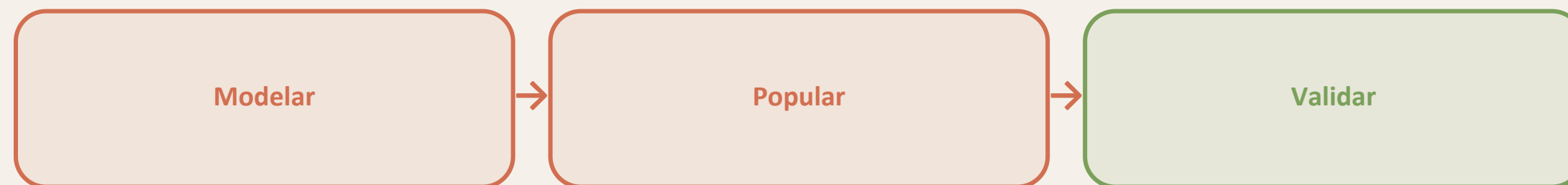
REGRA DE REDAÇÃO

- ✓ Cada análise cruza ≥ 2 fontes. Insight acionável no fim. R\$ em reais.



Bloco 3 *Supabase via MCP*. Banco real, em SQL.

CSV não tem tipo, não tem chave, não aguenta acesso concorrente. O banco resolve.



POR QUE MCP

- ✓ Claude Code fala SQL direto no Supabase — sem abrir o painel
- ✓ Criar tabela, rodar query, inspecionar schema em linguagem natural
- ✓ Pré-requisito: MCP do Supabase configurado ANTES da aula

PLANO B (SEM MCP)

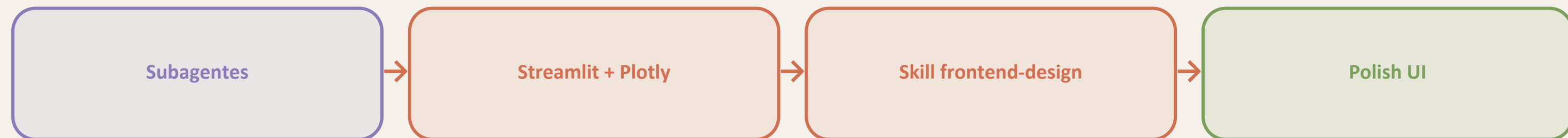
- ✓ Claude gera CREATE TABLE + INSERT → você cola no editor SQL do painel



Bloco 4 *Dashboard.*

Subagentes → Streamlit → polish.

Um diretor não abre o Supabase. Precisa de interface.



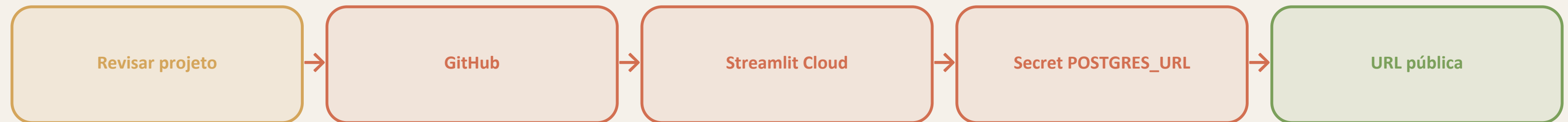
TRÊS PÁGINAS

- ✓ Vendas — KPIs (receita, total, ticket, clientes únicos) + tempo + top 5
- ✓ Clientes — receita por segmento + top 10
- ✓ Pricing — classificação vs concorrência + alertas
- ✓ Stack: Streamlit + Plotly + psycopg2 + python-dotenv
- ✓ Skill frontend-design eleva visual padrão do Streamlit



Bloco 5 *Deploy*. Streamlit Cloud em 3 passos.

Sobe pro GitHub, conecta na plataforma, cadastra secret, abre URL.



CHECKLIST PRÉ-DEPLOY

- ✓ requirements.txt cobre todas as deps
- ✓ .gitignore protege o .env
- ✓ Nenhuma credencial hardcoded no app.py
- ✓ POSTGRES_URL cadastrado como Secret na plataforma



Bloco 6 *Agente Telegram.* SQL via Claude, dado real.

Pipeline: Telegram → API Claude → SQL → Supabase → resposta formatada.



A CHAVE: CLAUDE NÃO INVENTA NÚMEROS

- ✓ Claude gera SQL → banco devolve dado real → Claude só formata
- ✓ Sem alucinação. Sem inventar. Schema vai no prompt.

SEGURANÇA

- ✓ Aceitar APENAS queries SELECT antes de executar
- ✓ Tratar erro de conexão e de SQL sem derrubar o bot



Demo. *E-commerce* do bruto à decisão.

Uma empresa abre operação digital. Quer decidir melhor com dados. 4 CSVs, 2 horas.

O QUE QUEREMOS CONSTRUIR

O cenário

- pedidos_vendas.csv (sujo) + 4 CSVs do case
- limpar.py reprodutível
- Análise: vendas, clientes, pricing
- Supabase com 4 tabelas relacionadas
- Dashboard publicado com 3 páginas
- Bot Telegram respondendo perguntas

POR QUE ESSE PROJETO

As camadas tocadas

- ✓ Pandas + Claude Code (limpeza)
- ✓ CLAUDE.md + .llm/ (contexto)
- ✓ Supabase MCP (banco em SQL natural)
- ✓ Streamlit + Plotly + skill UI (interface)
- ✓ Streamlit Community Cloud (deploy)
- ✓ API Claude + python-telegram-bot (conversacional)



A *estrutura* final entregue.

4 cases, 4 entregáveis. Todos rodam de novo amanhã.

AULA-CLAUDE-CODE-DADOS/

- ✓ database.md (schemas — fonte da verdade)
- ✓ case-00-limpeza/ → limpar.py, pedidos_limpos.csv
- ✓ case-01-analise/ → CLAUDE.md, .llm/, scripts + gráficos
- ✓ case-02-dashboard/ → CLAUDE.md, .claude/skills/, app.py, requirements.txt
- ✓ case-03-agente/ → CLAUDE.md, .llm/, agente.py, requirements.txt
- ✓
- ✓ 4 pastas, 4 entregáveis vivos. Tudo no Git, tudo reproduzível.



Faça. Evite.

As cicatrizes do workflow dirigido.

FAÇA SEMPRE

Hábitos que escalam

- ✓ Diagnóstico antes da correção (limpeza)
- ✓ CLAUDE.md + .llm/ em todo projeto
- ✓ Validação manual depois de cada passo
- ✓ Cada análise cruza ≥ 2 fontes
- ✓ Aceitar apenas SELECT no agente

EVITE SEMPRE

Hábitos que travam

- ✗ Confiar no output do Claude sem conferir
- ✗ Reexplicar contexto a cada sessão
- ✗ Análise solta em Jupyter sem virar produto
- ✗ Commitar .env no GitHub
- ✗ Deixar o agente rodar UPDATE/DELETE



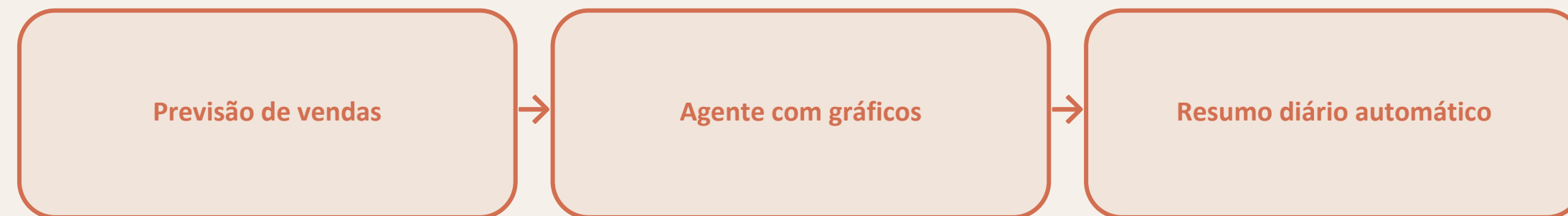
Quando esse workflow *não* serve.

Volume real-time, ML pesado, governança regulada — outro stack.



Pós-aula. *Desafios* extras.

Três extensões usando o mesmo workflow. Cada uma expande uma camada.



COMO CONTINUAR

- ✓ Esses 3 desafios usam exatamente o que você aprendeu hoje
- ✓ Cada um expande uma camada: ML, multimodal, scheduling
- ✓ Repositório fica aberto — me marca quando subir



O analista virou *diretor de orquestra*.

Próxima aula → Projeto 3: CRM completo com Kanban + Slack + Notion · Semana 7